

Insight 01 — Heat-stress exposure by year

Methodology, definitions, and formulas

2026-05-19

Overview

Insight 01 of the *Living through rising heat* site counts how much time per year a city spends above each of three felt-temperature thresholds (UTCI > 32, > 38, > 46 °C), plus how much time it spends in a *tropical-night* state (air temperature > 25 °C during night hours).

Results are reported in **day-equivalents**: total hours above the threshold divided by 24. A value of 10 means a cumulative $10 \times 24 = 240$ hours of exposure across the year, irrespective of which calendar days or hours those came from.

Data source

- **Provider:** Copernicus Climate Data Store, *Thermal Trace* service.
- **Variables:** `t2m` (air temperature at 2 m above ground) and `utci` (Universal Thermal Climate Index).
- **Resolution:** hourly samples, snapped to the underlying ERA5 ~0.25° grid cell containing the requested point.
- **Years:** 1950–2025 inclusive (76 years).
- **Endpoint:** `https://apps.climate.copernicus.eu/thermal-trace-server/aggregated-timeseries-new`
- **Parameters used:** `temporal_agg=hourly` , `r=0` , `apply_mask=false` , `start_time=<Y>-01-01` , `end_time=<Y>-12-31` . The `values_max_daily` array in the response is, at hourly aggregation, the per-hour reading itself (despite the field name).

Each year-file holds 8 760–8 784 hourly samples; a leap year contributes 24 extra samples. Files are saved as raw UTC JSON to `final-data/<city>-<year>-hourly-{t2m,utci}.json` .

Coordinates

The API snaps each requested (longitude, latitude) to the underlying ERA5 grid. The resolved cell centroids used here:

City	Longitude	Latitude
Ahmedabad	72.58	23.02
Ajmer	74.72	26.33

City	Longitude	Latitude
Bangalore	77.59	12.97
Chennai	80.27	13.08
Darjeeling	88.31	26.84
Delhi	77.21	28.61
Guwahati	91.75	26.18
Kodai	77.49	10.23
Mumbai	73.00	19.19
Munnar	77.06	10.09
Ooty	76.70	11.41
Shimla	77.17	31.10

Time-zone handling

The API returns hourly timestamps in **UTC**. India runs on **IST = UTC + 5:30**.

ERA5 samples land on the hour in UTC, so when re-interpreted as IST they sit at the half-past mark of each hour (UTC 00:00 → IST 05:30, UTC 01:00 → IST 06:30, ...). For each UTC sample index k in a year file:

IST clock hour at sample $k = (k + 5) \bmod 24$

So sample 0 (UTC 00:00 on Jan 1) is attributed to IST hour 5 (i.e. the 05:30 reading), sample 1 to IST hour 6, and so on. Sample 19 is the first that lands on IST 00:30 of Jan 2.

This shift matters only when we filter for *night hours* — see “Tropical nights” below.

Thresholds and definitions

Let:

- $T_2(k)$ = air temperature at 2 m, sample k , in °C
- $U(k)$ = UTCI value, sample k , in °C
- n = number of hourly samples in the year (8760 or 8784)
- $h(k) = (k + 5) \bmod 24$ = IST clock hour for sample k
- $\mathbf{1}\{\dots\}$ = indicator function (1 if condition holds, 0 otherwise)
- **Night window** in IST: $h \geq 19$ OR $h \leq 7$ (13 hours per IST day, from the 19:30 reading through the 07:30 reading)

All counts use **strict inequality** ($>$, not $\geq$). An exactly-threshold hour is **not** counted; impact at hourly resolution is negligible.

Strong heat stress

$$\text{strong_dayeq} = (1/24) \cdot \sum \text{over } k \, 1\{ U(k) > 32 \}$$

Count of hourly samples where UTCI exceeds 32 °C, divided by 24.

Very strong heat stress

$$\text{very_strong_dayeq} = (1/24) \cdot \sum \text{over } k \, 1\{ U(k) > 38 \}$$

Extreme heat stress

$$\text{extreme_dayeq} = (1/24) \cdot \sum \text{over } k \, 1\{ U(k) > 46 \}$$

Tropical nights (*actual temperature, not UTCI*)

$$\text{tropical_night_dayeq} = (1/24) \cdot \sum \text{over } k \, 1\{ T_2(k) > 25 \} \cdot 1\{ h(k) \in \text{night} \}$$

Both conditions must hold: it's an IST night hour AND the air temperature exceeds 25 °C.

The night-window restriction means the maximum theoretical value is $13/24 \times 365 \approx 197.7$, not 365.

The tropical-nights panel uses **T2M (air temperature)** because that is the conventional meteorological definition. The other three panels use UTCI (*felt* temperature) because they describe heat *stress on the body* rather than ambient weather.

T2M overlay (dotted lines, optional)

For visual comparison, each heat-stress panel can be overlaid with the same threshold applied to T2M instead of UTCI:

$$\text{strong_t2m_dayeq} = (1/24) \cdot \sum \text{over } k \, 1\{ T_2(k) > 32 \}$$

$$\text{very_strong_t2m_dayeq} = (1/24) \cdot \sum \text{over } k \, 1\{ T_2(k) > 38 \}$$
$$\text{extreme_t2m_dayeq} = (1/24) \cdot \sum \text{over } k \, 1\{ T_2(k) > 46 \}$$

The gap between the solid UTCI line and the dotted T2M line at the same threshold is the contribution of humidity, wind and radiation to perceived heat stress. A positive gap (UTCI > T2M) means felt heat amplifies above recorded air temperature; a near-zero gap means the two metrics agree.

The dotted overlay is omitted on the *tropical nights* panel because that panel is already a T2M metric.

Output file

For each city the script writes `exposure-data/<city>-exposure-by-year.csv` with these columns:

Column	Definition
<code>year</code>	Calendar year, integer
<code>strong_dayeq</code>	UTCI > 32 — formula above
<code>very_strong_dayeq</code>	UTCI > 38
<code>extreme_dayeq</code>	UTCI > 46
<code>tropical_night_dayeq</code>	T2M > 25 during IST night hours
<code>strong_t2m_dayeq</code>	T2M > 32 (overlay)
<code>very_strong_t2m_dayeq</code>	T2M > 38 (overlay)
<code>extreme_t2m_dayeq</code>	T2M > 46 (overlay)

Values are rounded to 4 decimal places.

Implementation

Two scripts ship with the data:

- `fetch_all_years.py` — pulls hourly JSON from the Copernicus API for every (city, year, variable) combination. Supports parallel requests (`--concurrency N`), per-job progress logging, and resumable fetches (`--skip-existing`).
- `compute_exposure_csvs.py` — reads the raw UTC JSON and emits the per-city CSVs described above. Idempotent.

Reproducing

From the project root:

```
# fetch all hourly data (~ 1 800 API calls, ~1.5–2 h at concurrency 4)  
python3 fetch_all_years.py --concurrency 4  
  
# compute exposure CSVs (seconds, not minutes – pure local computation)  
python3 compute_exposure_csvs.py
```